PRACTICAL 6

Date: 21-09-2026

Program 1: Smart Device Controller

Aim: To create a Switchable interface containing on(), off(), and a default toggle() method. Implement it in Fan and Lamp classes, toggle a collection of devices, and demonstrate a functional permission interface using both an anonymous class and a lambda expression.

Code:

```
public class SmartController { public static void
    main(String[] args) { Switchable[] items = {new
    Fan(), new Lamp()}; for (Switchable item :
    items) { item.toggle();
        }
        PowerPermission rule1 = new PowerPermission() {
            public boolean allowed(String device, int hour)
            { return hour >= 7 && hour <= 22;
            }
        };
        System.out.println("Fan at 8: " + rule1.allowed("Fan", 8));
        System.out.println("Lamp at 5: " + rule1.allowed("Lamp", 5));
        PowerPermission rule2 =
                (device, hour) -> hour >= 7 && hour <= 22;
        System.out.println("Lambda check:");
        System.out.println("Fan at 8: " + rule2.allowed("Fan", 8));
        System.out.println("Lamp at 5: " + rule2.allowed("Lamp", 5));
    }
} interface Switchable {
void on(); void off();
default void toggle() {
on();
    }
```

```
}    class    Fan    implements
Switchable { public void on() {
        System.out.println("Fan started.");
    }    public    void
    off() {
        System.out.println("Fan stopped.");
    }
}    class    Lamp    implements
Switchable { public void on() {
        System.out.println("Lamp switched on.");
    }    public    void
    off() {
        System.out.println("Lamp switched off.");
    }    }@FunctionalInterface    interface
PowerPermission  {  boolean  allowed(String
device, int hour);
}
```

OUTPUT:

```
Fan started.
Lamp switched on.
Fan at 8: true
Lamp at 5: false
Lambda check:
Fan at 8: true
Lamp at 5: false

=== Code Execution Successful ===
```

Learning Outcome:

• Understand interfaces and default methods in Java.

• Implement an interface in multiple device classes and override its methods.

• Use an array of interface references and iterate through it.

• Create a functional interface and implement it with an anonymous class and a lambda.

Program 2: Multi-channel Notification System

Aim: To define a functional Notifier interface, create email and SMS senders with lambda expressions, broadcast a common message, and send urgent notifications twice by using a marker interface.

Code:

```
public class AlertBroadcast { public static

    void main(String[] args) {

        Notifier email = text ->

            System.out.println("Email alert: " + text);

        Notifier sms = text ->

            System.out.println("SMS alert: " + text);

        Notifier prioritySms = new PriorityNotifier(sms);

        Notifier[] channels = {email, prioritySms};

        String alert = "Lab submission closes

        tomorrow."; System.out.println("Sending

        alerts:"); for (Notifier channel : channels) {

        channel.send(alert); if (channel instanceof

        Urgent) { channel.send(alert);

            }

        }

    }

}

@FunctionalInterface

interface Notifier { void

send(String text);

}       interface

Urgent {

}   class   PriorityNotifier   implements   Notifier,

Urgent { private Notifier channel;

    PriorityNotifier(Notifier channel) {

        this.channel = channel;
```

```
    } public void send(String

    text) { channel.send(text);

    }

}
```

OUTPUT:

```
Sending alerts:
Email alert: Lab submission closes tomorrow.
SMS alert: Lab submission closes tomorrow.
SMS alert: Lab submission closes tomorrow.

=== Code Execution Successful ===
```

Learning Outcome:

• Understand functional interfaces and marker interfaces.

• Implement email and SMS notifications using lambda expressions.

• Store different notification channels in an array.

• Broadcast a message using a loop.

• Use instanceof to detect urgent senders and repeat their notification.

Program 3: Runtime Discount Calculator

Aim: To build a discount calculator using a functional interface and lambda expressions. Allow the user to select a discount rule at runtime and apply it to a collection of prices.

Code:

```
import  java.util.Scanner;  public  class  Main  {

public  static  void  main(String[]  args)  {  Scanner

input = new Scanner(System.in); double[] amounts =

{1200, 2500, 3500, 4800};

        System.out.println("Select a discount:");

        System.out.println("1. 5 percent");

        System.out.println("2. 15 percent");
```

```
        System.out.println("3. Flat Rs. 150");
        System.out.print("Choice:    ");    int
        choice = input.nextInt(); DiscountRule
        selected; if (choice == 1) { selected
        = amount -> amount * 0.95;
        } else if (choice == 2) { selected =
            amount -> amount * 0.85;
        } else if (choice == 3) { selected = amount ->
            Math.max(0, amount - 150);
        } else {
            System.out.println("Invalid
            selection."); input.close(); return;
        }
        System.out.println("Discounted     amounts:");
        for (double amount : amounts) {
            System.out.println(
                "Rs. " + amount + " -> Rs. " +
                selected.calculate(amount)
            ); }
        input.clo
        se();
    }
}
@FunctionalInterface      interface
DiscountRule        {        double
calculate(double amount);
}
```

OUTPUT:

```
Select a discount:
1. 5 percent
2. 15 percent
3. Flat Rs. 150
Choice: 1
Discounted amounts:
Rs. 1200.0 -> Rs. 1140.0
Rs. 2500.0 -> Rs. 2375.0
Rs. 3500.0 -> Rs. 3325.0
Rs. 4800.0 -> Rs. 4560.0

=== Code Execution Successful ===
```

Learning Outcome:

- Understand functional interfaces and lambda-based rules.
- Select a discount operation based on user input.
- Apply the chosen rule to multiple prices using a loop.
- Recognize how Java packages and runnable JAR files can be used to organize and distribute an application.

Note: The program logic and learning objectives are retained, while class names, examples, values, prompts, and descriptions have been revised.